

Danmarks
Tekniske
Universitet



42114 Integer Programming Project Assignment

AUTHORS

Ifigeneia Tziola - s222569
Charalampos Kozaris - s230224
Antonios Kostoudis - s233560

November 5, 2023

Contents

1	Crew Dispatching	1
2	Crew Work Assignment	2
2.1	Dynamic Programming	2
2.2	Integer Programming	4
3	Code	7

1 Crew Dispatching

The assignment's example problem is presented in Figure 1.

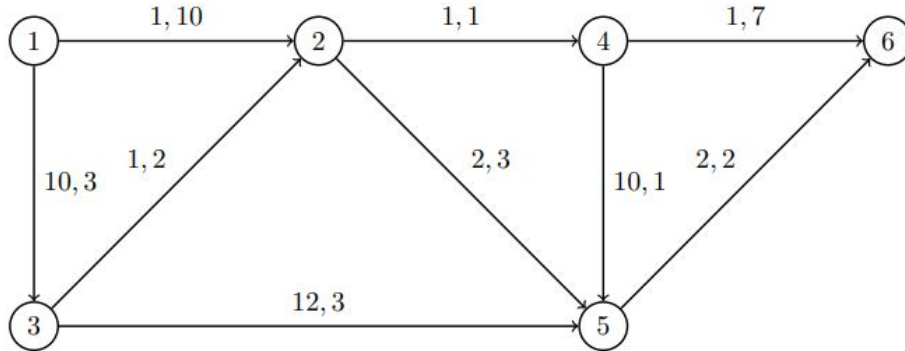


Figure 1: Example Problem. Values on the edges are $c_{i,j}, t_{i,j}$

The manager's model is as follows:

$$\min = \sum_{(i,j) \in A} c_{ij} x_{ij} \quad (1)$$

$$\sum_{(i,j) \in A} t_{ij} x_{ij} \leq T \quad (2)$$

$$\sum_{(s,j) \in A} x_{sj} = 1 \quad (3)$$

$$\sum_{(i,d) \in A} x_{id} = 1 \quad (4)$$

$$x_{ij} \in \{0, 1\} \text{ for } (i, j) \in A$$

Question 1

The manager's model is missing constraints that make sure that whenever a node is used by the path it has exactly one entering edge and one leaving edge. The following constraints are added to the model to address this issue:

$$\sum_{i \in V} x_{ij} \leq 1 \quad \forall j \in V \quad (5)$$

$$\sum_{j \in V} x_{ij} \leq 1 \quad \forall i \in V \quad (6)$$

Equation 5 ensures that every node j can have no more than one entering edge, while Equation 6 ensures that every node i can have no more than one departing edge. To ensure that each node that has an entering edge also has a departing edge, Equation 7 is added to the model:

$$\sum_{i \in V} x_{ij} - \sum_{j \in V} x_{ji} = 0 \quad j \neq \{s, d\} \quad (7)$$

$$x_{ij} \in \{0, 1\} \text{ for } (i, j) \in A$$

Question 2

The LP model was entered into a solver and an optimal solution was found. For a time limit of $T = 10$ the optimal solution is the itinerary $1 \rightarrow 3 \rightarrow 2 \rightarrow 5 \rightarrow 6$ with a total cost of 15 that takes 10 hours to complete.

Question 3

To make the Lagrangian relaxation of the binary programming model the time constraint in Equation 2 is dropped from the model and is added in the objective function in Equation 1 in the form of a penalty term:

$$\min = \sum_{(i,j) \in A} c_{ij}x_{ij} - u(T - t_{ij}x_{ij}) \quad (8)$$

This change means that there is no time constraint imposed on the model and that the new term acts as a counter-motive for the solver to suggest costly solutions, causing an increase in the objective value for greater travel times. The sign used for u is such that the term is activated for positive values of u when the constraint in Equation 2 is violated.

Question 4

For $u = 0$ the cheapest path has an itinerary of $1 \rightarrow 2 \rightarrow 4 \rightarrow 6$ with a total cost of 3 that takes 18 hours to complete. This happens because the penalty term that includes the time limit is not taken into consideration in the objective function for $u = 0$ and the solver only tries to minimize the cost. In general, when u is increased the penalty term in the objective function gains more weight, meaning that the violation of the time constraint affects the objective value more. This leads to solutions of increased costs and shorter travel times.

Question 5

Solving the model for multiple values of u the following table is formed:

Range of u	Path Cost $\sum c_{ij}$	Travel Time $\sum t_{ij}$	$z^L(u)$
$0 \leq u \leq 0.6$	3	18	$3 \leq z^L(u) \leq 7.8$
$0.7 \leq u \leq 2$	5	15	$8.5 \leq z^L(u) \leq 15$
$2.1 \leq u \leq 4.4$	15	10	15
$4.5 \leq u$	24	8	$z^L(u) \leq 15$

Table 1: Dual Bound Table

The best dual bound that can be achieved should not exceed the primal objective. In fact, it should be the highest possible value before the primal objective starts to increase due to the penalty for time constraint violations, which is $z^L(u) = 15$ for $2.1 \leq u \leq 4.4$. Therefore, the best dual bound that can be achieved, according to the table, is 15.

Question 6

Solving the original problem, the optimal solution is a total cost of 15. In the case of a Lagrangian Relaxation for a minimization problem, the dual value is expected to be less or equal than the optimal solution of the original problem. In this case, the value of the Lagrangian Dual as it is shown in Table 1 is equal to the optimal solution of the original problem, so it does not come as a surprise.

2 Crew Work Assignment

2.1 Dynamic Programming

For task i , s_i denotes the time it takes for the task to be completed while k is the number of teams that the tasks can be assigned to:

$$S = \{100, 200, 400, 500, 900, 700\}, k = 3$$

Question 7

Since the first divider is placed between task 1 and task i and the second divider is placed between task i and $i+1$ then the cost of the last assignment is:

$$\sum_{j=i+1}^n = S_j$$

Question 8

To compute the values of the base cases we can use the recurrence formula. The base cases are:

$$\begin{aligned} C(1, j) \text{ for } 1 \leq j \leq k & \text{ (one task assigned to } k \text{ possible assignments),} \\ C(i, 1) \text{ for } 1 \leq i \leq n & \text{ (} n \text{ tasks assigned to a single assignment)} \end{aligned}$$

For $C(1, j)$ all values generated by the recursion are the same ($C(1, 1) = C(1, 2) = C(1, 3) = 100$) because in every case Task 1 is assigned to one or more assignments. For $C(i, 1)$ we assign all the tasks in a single assignment for every i , so that $C(1, 1) = C_1 = 100$, $C(2, 1) = C_1 + C_2 = 300$, ... $C(6, 1) = C_1 + C_2 + C_3 + C_4 + C_5 + C_6 = 2800$.

Since $1 \leq i \leq n$ and $1 \leq j \leq k$ and $n = 6$, $k = 3$ the computed values for all $C(i, j)$ which will be the entries in the table will be 18 in total. Simply put, we have three cases (one for each number of assignments) for each number of tasks. The optimal solution will be found in the bottom right of the table, where six tasks are assigned to three assignments ($C(6, 3)$).

In the worst case one entry will depend at the most on two previous entries. For example, $C(6, 3)$ depends on entry $C(5, 2)$ which in turn depends on entry $C(4, 1)$. In general, for k assignments, one entry will depend at the most on $k - 1$ entries.

Question 9

Below we show an example of how we made the calculations of the entries of the table.

$$C(6, 3) = \min \left\{ \begin{array}{l} \max \{C(1, 2), c_2 + c_3 + c_4 + c_5 + c_6\} = \max\{100, 2700\} = 2700 \\ \max \{C(2, 2), c_3 + c_4 + c_5 + c_6\} = \max\{200, 2500\} = 2500 \\ \max \{C(3, 2), c_4 + c_5 + c_6\} = \max\{400, 2100\} = 2100 \\ \max \{C(4, 2), c_5 + c_6\} = \max\{1200, 1600\} = 1600 \\ \max \{C(5, 2), c_6\} = \max\{1200, 700\} = \mathbf{1200} \end{array} \right\} = 1200$$

Following the recurrence formula we get all the entries of Table 2:

	Assignments		
	1	2	3
1	100	100	100
2	300	200	200
3	700	400	400
4	1200	700	500
5	2100	1200	900
6	2800	1600	1200

Table 2: Dynamic Programming Recurrence Values.

Question 10

After the dynamic programming table has been filled out we have determined that $C(n, k)$ is the optimal value (where $n=6$ and $k=3$), which can be found on the bottom right of the table. To reconstruct the optimal solution we have to backtrack through the iterations:

We start with the entry on the bottom right corner of the table and we look at all the different possible values of that entry. We spot the minimum solution.

$$C(n, k) = \min \left\{ \begin{array}{l} \max \{C(1, k-1), c_{n-4} + c_{n-3} + c_{n-2} + c_{n-1} + c_n\} \\ \max \{C(2, k-1), c_{n-3} + c_{n-2} + c_{n-1} + c_n\} \\ \max \{C(n-3, k-1), c_{n-2} + c_{n-1} + c_n\} \\ \max \{C(n-2, k-1), c_{n-1} + c_n\} \\ \max \{C(n-1, k-1), c_n\} \end{array} \right\}$$

For the sake of this example let's assume that it is value: $\max \{C(n-1, k-1), c_n\}$. In that case, we go backward again to:

$$C(n-1, k-1) = \min \left\{ \begin{array}{l} \max \{C(1, k-2), c_{n-4} + c_{n-3} + c_{n-2} + c_{n-1}\} \\ \max \{C(2, k-2), c_{n-3} + c_{n-2} + c_{n-1}\} \\ \max \{C(n-3, k-2), c_{n-2} + c_{n-1}\} \\ \max \{C(n-2, k-2), c_{n-1}\} \end{array} \right\}$$

Again, for the sake of the example, let's assume that the minimum solution is: $\max \{C(n-2, k-2), c_{n-1}\}$.

We follow the same procedure until the different possible values of an entry have the term $C(i, 1)$ in the formula. In our case, since the group assignments are $k=3$, we have already identified the path to the optimal solution and therefore we can stop the backtracking. If that was not the case, we would have continued the iterations until an entry would have the term $C(i, 1)$ in the formula.

In our example that procedure is giving: $C(n, k) \Rightarrow C(n-1, k-1) \Rightarrow C(n-2, k-2)$ as the optimal solution.

2.2 Integer Programming

Question 11

Let's define the original and the relaxed problems as follows respectively:

$$\begin{aligned} z &= \min \{c(x) : x \in X \subseteq R^n\} \\ z^R &= \min \{f(x) : x \in T \subseteq R^n\} \end{aligned}$$

All the feasible solutions to the original problem are also solutions to the newly defined, "relaxed" problem ($X \subseteq T$). Furthermore, all the solutions to the original problem result in the same optimal value in the relaxed problem ($c(x) = f(x)$). Therefore, the new problem is a relaxation of the original problem.

Question 12

Below a simple greedy heuristic will be described:

- **Step 1:** Sort all 6 tasks in decreasing order.
- **Step 2:** Create 3 empty lists, one for each group of assignments.
- **Step 3:** Now iterate through the sorted tasks. Every time assign the task to the list with the lowest total time at the moment of the assignment. In case there are more than one groups with equal lowest total times, assign the task at hand to the first group.
- **Step 4:** Update the total time for the chosen group.
- **Step 5:** Iterate until all tasks are assigned to groups.
- **Step 6:** Now calculate the max. total time between all 3 groups.

Using the greedy heuristic we calculate that the three assignments have the following durations:

$$\{1, 2, 3\} = \{1000, 900, 900\}$$

The greedy heuristic provides us with the value 1000 which is an UB and a primal bound to the minimization problem.

Question 13

The integer programming model uses a binary variable x_{ip} which takes the value of 1 if task i is in assignment p and 0 otherwise.

To ensure that each task is assigned to exactly one assignment and that each assignment contains at least one task the following constraints are introduced respectively:

$$\sum_{p=1}^k x_{ip} = 1 \quad \forall i \in \{Task_1, Task_2, Task_3, Task_4, Task_5, Task_6\}$$

$$\sum_{i=1}^n x_{ip} \geq 1 \quad \forall p$$

The duration of each assignment p can be defined as:

$$\sum_{i=1}^n x_{ip} s_i \quad \forall p$$

Since the assigned tasks do not have to be consecutive, we can set the first assignment to have the longest duration:

$$\sum_{i=1}^n x_{i1} s_i \geq \sum_{i=1}^n x_{ip} s_i \quad \forall p$$

Now the objective function can be defined as:

$$\min \sum_{i=1}^n x_{i1} s_i$$

The model was implemented in Julia and the results are presented in Table 3. The optimal solution is a longest assignment with a duration of 1000 minutes. The same value was the upper bound obtained with the greedy heuristic in the previous question.

Assignment	Tasks	Cost
1	1,5	1000
2	2,6	900
3	3,4	900

Table 3: Assignments with non-consecutive tasks

Question 14

Shifting back to the problem where assignments can only consist of consecutive tasks, matrix D contains W feasible assignments i as columns and n tasks as rows j . The cost of each possible assignment is denoted as c_i . The binary variable x_i is defined as taking the value 1 if assignment i is part of the solution and the value 0 otherwise. The following constraint ensures that each task needs to be part of the solution:

$$\sum_{i=1}^W D_{ij} x_i = 1 \quad \forall j$$

To ensure that exactly k assignments are created the following constraint is introduced:

$$\sum_{i=1}^W x_i = k$$

In order to create an objective function that minimizes the longest assignment, an auxiliary variable M is introduced and is set to be greater or equal to each of the assignments' durations:

$$M \geq x_i c_i \quad \forall i$$

Now the objective function can be defined as:

$$\min M$$

The model was implemented in Julia and the optimal solution is presented in Table 4:

Assignment	Tasks	Cost
4	1,2,3,4	1200
19	5	900
21	6	700

Table 4: Assignments with consecutive tasks

The longest assignment has a duration of 1200 minutes. This is the same result as the one obtained through the recursion in Question 9.

The disadvantage is that if this approach is that if there are n tasks the dimensions of matrix D will be $n \times \frac{n(n+1)}{2}$. This makes it increasingly difficult to input matrix D as a parameter in a model with e.g. 100 tasks where 505.000 entries would be needed.

3 Code

Listing 1: Question 4 - Crew Dispatching

```

1  import Pkg
2  using JuMP, Gurobi, Printf
3
4
5  n = collect(1:6)
6  M = 1000
7  T = 10
8  restricted_n = collect(2:5)
9  u = 1
10
11
12 #Declare cost and time sets
13 c = [M 1 10 M M M;
14      M M M 1 2 M;
15      M 1 M M 12 M;
16      M M M M 10 1;
17      M M M M M 2;
18      M M M M M M]
19
20 t = [0 10 3 0 0 0;
21      0 0 0 1 3 0;
22      0 2 0 0 3 0;
23      0 0 0 0 1 7;
24      0 0 0 0 0 2;
25      0 0 0 0 0 0]
26
27
28 #Declare Gurobi model
29 m = Model(Gurobi.Optimizer)
30
31 #Definition of variables
32 @variable(m, x[i in n, j in n], Bin)
33 #@variable(m, u[i in u_set], Int)
34
35 #Objective Function
36 @objective(m, Min, sum(c[i,j]*x[i,j] - u*(T - t[i,j]*x[i,j]) for i in n for j in n))
37
38 #Manager's Constraints
39 #@constraint(m, sum(t[i,j]*x[i,j] for i in n for j in n) <= T)
40 @constraint(m, sum(x[1,j] for j in n) == 1)
41 @constraint(m, sum(x[i,6] for i in n) == 1)
42
43 #Added Constraints
44 @constraint(m, [j in n], sum(x[i,j] for i in n) <= 1)
45 @constraint(m, [i in n], sum(x[i,j] for j in n) <= 1)
46 @constraint(m, [j in restricted_n], sum(x[i,j] for i in n) - sum(x[j,i] for i in n) == 0)
47
48 optimize!(m)
49
50
51 if termination_status(m) == MOI.OPTIMAL
52     println("Optimal solution found")
53

```

```

54     println("itinerary: $(value(x))")
55     #println()
56     #for i in n
57         # for j in n
58             #println("itinerary $(value(x[i,j]))")
59         #end
60     #end
61     println("times: $(sum(value(x[i,j]) * t[i,j] for i in n, j in n))")
62     @printf "\nObjective value: %0.3f\n\n" objective_value(m)
63     @printf "\nActual itinerary cost: %0.3f\n\n" value(sum(c[i,j]*x[i,j] for i in n for j in n))
64 else
65     error("No solution.")
66 end

```

Listing 2: Question 13 - Non-Consecutive Tasks

```

1
2 using JuMP, Gurobi
3
4
5 # Number of tasks
6 n = 6
7
8 # Number of assignments
9 k = 3
10
11 # Duration of tasks
12 S = [100, 200, 400, 500, 900, 700]
13
14 model = Model(Gurobi.Optimizer)
15
16 @variable(model, x[1:n, 1:k], Bin)
17
18 @objective(model, Min, sum(x[i,1]*S[i] for i=1:n))
19
20 @constraint(model, [i=1:n], sum(x[i,p] for p=1:k) == 1)
21 @constraint(model, [p=1:k], sum(x[i,p] for i=1:n) >= 1)
22
23 @constraint(model, [p=1:k], sum(x[i,1]*S[i] for i=1:n) >= sum(x[i,p]*S[i] for i=1:n))
24
25
26
27
28 JuMP.optimize!(model)
29
30 if termination_status(model) == MOI.OPTIMAL
31     println("Objective is: ", JuMP.objective_value(model))
32     println("Solution is: ")
33     tasks = Vector{Vector{Int64}}(undef, k)
34     for p=1:k
35         tasks[p] = []
36         for i=1:n
37             if (JuMP.value(x[i,p]) > 0.5)
38                 push!(tasks[p], i)
39             end
40         end
41         print("Assignment ", p, ": ", tasks[p])
42         println()

```

```

43     end
44 else
45     println("Optimise was not successful. Return code", termination_status(model))
46 end

```

Listing 3: Question 14 - Consecutive Tasks

```

1  using JuMP, GLPK, Gurobi
2
3
4
5  # Number of assignments in the matrix
6  W= 21
7
8  # Number of assignments in the Solution
9  k = 3
10
11 # Number of tasks in the problem
12 t = 6
13
14 # Matrix is column by column, so first index is column and second index is row
15 D = [1 1 1 1 1 1 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0;
16      0 1 1 1 1 1 1 1 1 1 1 0 0 0 0 0 0 0 0 0 0;
17      0 0 1 1 1 1 0 1 1 1 1 1 1 1 1 0 0 0 0 0 0;
18      0 0 0 1 1 1 0 0 1 1 1 0 1 1 1 1 1 1 0 0 0;
19      0 0 0 0 1 1 0 0 0 1 1 0 0 1 1 0 1 1 1 1 0;
20      0 0 0 0 0 1 0 0 0 0 1 0 0 0 1 0 0 1 0 1 1]
21
22 # Cost vector
23 c = [100 300 700 1200 2100 2800 200 600 1100 2000 2700 400 900 1800 2500 500 1400 2100 900 1600 700]
24
25 model = Model(Gurobi.Optimizer)
26
27 @variable(model, x[1:W], Bin)
28
29 # Variable to act as equal to the longest assignment
30 @variable(model, M)
31
32
33 # YOUR MODEL IN HERE
34 @objective(model, Min, M)
35
36 @constraint(model, [j=1:t], sum(D[j,i]*x[i] for i in 1:W) == 1)
37 @constraint(model, sum(x[i] for i in 1:W) == k)
38
39 # Setting variable M greater or equal than longest assignment
40 @constraint(model, [i=1:W], M >= x[i]*c[i])
41
42
43
44
45 # -----
46
47 JuMP.optimize!(model)
48
49 if termination_status(model) == MOI.OPTIMAL
50     println("Objective is: ", JuMP.objective_value(model))
51     #println("Objective is: ", sum(JuMP.value(x[i])*c[i] for i=1:W))

```

```
52     println("Solution is: ")
53     for i=1:W
54         if (JuMP.value(x[i]) > 0.5)
55             print(i, ": ")
56             for j=1:t
57                 if (D[j,i]==1)
58                     print(j," ")
59                 end
60             end
61             println()
62         end
63     end
64 else
65     println("Optimise was not successful. Return code", termination_status(model))
66 end
```